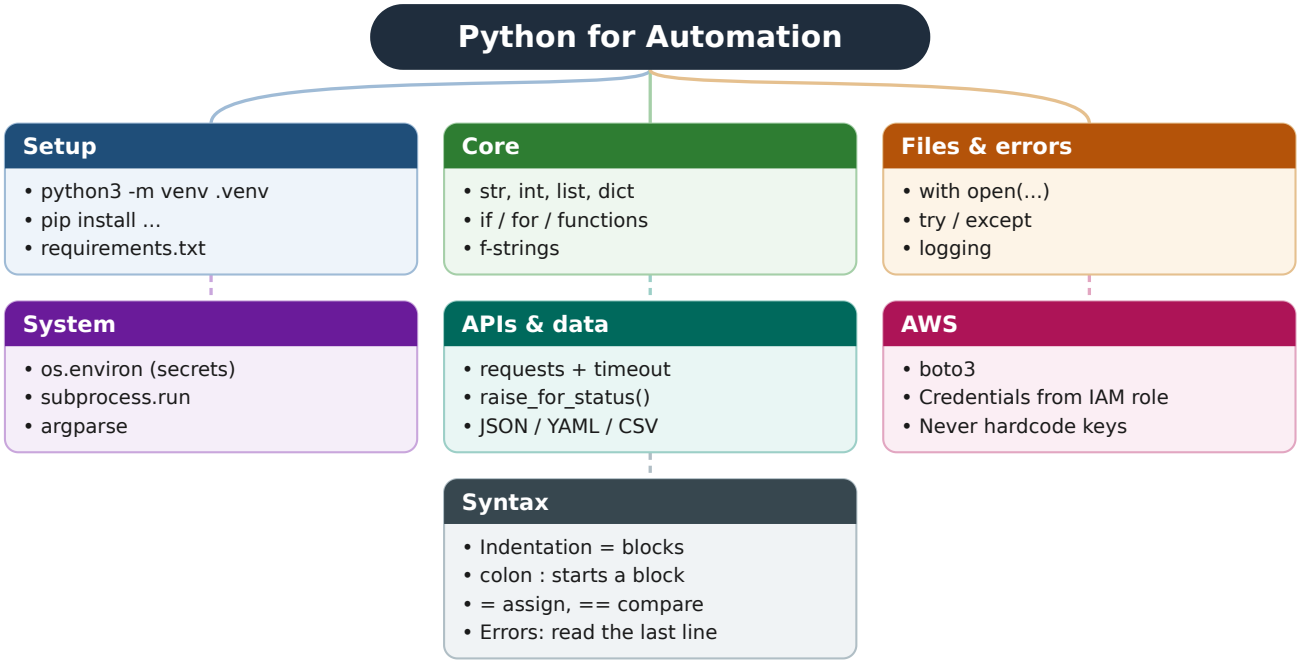
6. Python for Automation

In this section: 6.1 Setup · 6.2 Basics · 6.3 Files, errors, environment variables · 6.4 APIs and JSON · 6.5 AWS with boto3 · 6.6 Structure of a real automation script · 6.7 Python basics: what is what



6.1 Setup

Command	What it does
`python3 --version`	Check the version
`python3 -m venv .venv`	Create a virtual environment (isolated packages per project)
`source .venv/bin/activate`	Activate it (`deactivate` to leave)
`pip install requests boto3`	Install packages
`pip freeze > requirements.txt`	Save dependencies
`pip install -r requirements.txt`	Install saved dependencies

6.2 Basics

```
name = "web1"                       # string
port = 8080                         # integer
servers = ["web1", "web2"]          # list
config = {"env": "dev", "port": 80}  # dictionary (key → value)

for s in servers:
    if s == "web1":
        print(f"{s} is primary")   # f-string
    else:
        print(s)

def is_port_valid(p: int) -> bool:
    return 1 <= p <= 65535
```

6.3 Files, errors, environment variables

```
import os, logging, subprocess

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")

api_token = os.environ.get("API_TOKEN")      # secret comes from the environment
if not api_token:
    raise SystemExit("API_TOKEN is not set")

try:
    with open("/var/log/app.log") as f:      # 'with' closes the file automatically
        errors = [line for line in f if "ERROR" in line]
    logging.info("Found %d errors", len(errors))
except FileNotFoundError:
    logging.error("Log file not found")

result = subprocess.run(["df", "-h"], capture_output=True, text=True, check=True)
print(result.stdout)
```

6.4 APIs and JSON

```
import json, requests

resp = requests.get("https://api.github.com/repos/kubernetes/kubernetes", timeout=10)
resp.raise_for_status()                     # error on 4xx/5xx
data = resp.json()                          # JSON → Python dictionary
print(data["stargazers_count"])

with open("report.json", "w") as f:
    json.dump({"stars": data["stargazers_count"]}, f, indent=2)
```

6.5 AWS with boto3

```
import boto3   # uses credentials from AWS CLI config / IAM role, never hardcoded

ec2 = boto3.client("ec2", region_name="us-east-1")
for r in ec2.describe_instances()["Reservations"]:
    for i in r["Instances"]:
        print(i["InstanceId"], i["State"]["Name"])
```

Good habits: virtual environments, `requirements.txt`, `timeout=` on network calls, `try/except` with clear messages, logging instead of print in real tools, secrets from environment or a secrets manager.

6.6 Structure of a real automation script

```
#!/usr/bin/env python3
# Check that a list of URLs answers with HTTP 200.
import argparse, logging, sys
import requests

def check(url: str) -> bool:
    try:
        return requests.get(url, timeout=5).status_code == 200
    except requests.RequestException as e:
        logging.error("%s failed: %s", url, e)
        return False

def main() -> int:
    parser = argparse.ArgumentParser(description="URL health checker")
    parser.add_argument("urls", nargs="+", help="URLs to check")
    args = parser.parse_args()
    logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
    results = {u: check(u) for u in args.urls}
    for u, ok in results.items():
        logging.info("%s %s", "UP  " if ok else "DOWN", u)
    return 0 if all(results.values()) else 1        # exit code for CI/cron

if __name__ == "__main__":                          # run only when executed directly
    sys.exit(main())
```

Run: `python3 check.py https://example.com https://api.example.com`. Data formats you will parse daily: **JSON** (`json` module), **YAML** (`pip install pyyaml` → `yaml.safe_load`), **CSV** (`csv` module).

6.7 Python basics: what is what

Data types

Type	Example	What it is
`str` (string)	`"web1"` or `'web1'`	Text, always in quotes
`int`	`8080`	Whole number
`float`	`99.5`	Decimal number
`bool`	`True` / `False`	Yes/no (capital first letter)
`None`	`None`	"No value"
`list`	`["web1", "web2"]`	Ordered, changeable collection; index starts at 0
`tuple`	`("10.0.0.1", 22)`	Ordered, **unchangeable** collection
`dict`	`{"name": "web1", "port": 80}`	Key → value pairs
`set`	`{"dev", "prod"}`	Unique items, no order

Check a type with `type(x)`. Convert with `int("80")`, `str(80)`, `float("1.5")`, `list(...)`.

Strings

```
name = "web-server"
env = 'prod'
multi = """line one
line two"""                        # triple quotes = multi-line string

print(f"{name} runs in {env}")   # f-string: put variables inside {}
print(name.upper())              # WEB-SERVER
print("  text  ".strip())        # "text"  (remove spaces at both ends)
print(name.replace("-", "_"))    # web_server
print(name.split("-"))           # ['web', 'server']   (string → list)
print(",".join(["a", "b"]))      # a,b                 (list → string)
print(name.startswith("web"))    # True
print("server" in name)          # True  (contains?)
print(len(name))                 # 10    (length)
print(name[0], name[-1], name[0:3])   # w r web   (index and slicing)
```

Numbers and operators

Operators	Meaning	Example
`+ - * /`	Math (`/` always gives a float)	`7 / 2` → `3.5`
`// % **`	Whole division, remainder, power	`7 // 2` → 3, `7 % 2` → 1, `2 ** 3` → 8
`== != > < >= <=`	Comparison → True/False	`cpu > 80`
`and or not`	Logic	`cpu > 80 and env == "prod"`
`in` , `not in`	Membership	`"web1" in servers`
`=` vs `==`	Assign vs compare	`x = 5` sets, `x == 5` tests

Lists and dictionaries

```
servers = ["web1", "web2"]
servers.append("web3")           # add to the end
servers.remove("web1")           # remove by value
print(servers[0], len(servers))  # web2 2
for s in servers:                # loop over items
    print(s)

server = {"name": "web1", "port": 80}
print(server["name"])            # web1  (KeyError if missing)
print(server.get("env", "dev"))  # dev   (safe: default if missing)
server["env"] = "prod"           # add / update a key
for key, value in server.items():
    print(key, value)

ports = [p * 10 for p in range(1, 4)]   # list comprehension → [10, 20, 30]
```

Control flow and functions

```
cpu = 85
if cpu > 90:                    # colon : starts a block
    status = "critical"         # INDENTATION (4 spaces) defines the block
elif cpu > 70:
    status = "warning"
else:
    status = "ok"

for i in range(3):              # 0, 1, 2
    if i == 1:
        continue                # skip to the next loop
    print(i)

attempts = 0
while attempts < 3:             # repeat while the condition is True
    attempts += 1               # same as attempts = attempts + 1

def check_port(host: str, port: int = 22) -> bool:   # parameters, default value, type hints
    """Docstring: explains what the function does."""
    return port in (22, 80, 443)                     # return sends a value back

print(check_port("web1"), check_port("web1", 8080))   # True False
```

Modules, input and output

```
import os                       # standard library module
from pathlib import Path        # import one thing from a module

home = os.environ.get("HOME")   # read an environment variable
path = Path("/tmp") / "report.txt"
path.write_text("hello\n")      # write a file
print(path.read_text())         # read a file
```

Common errors and what they mean

Error	Meaning
`IndentationError`	Spaces/tabs wrong in a block
`SyntaxError`	Missing `:`, bracket or quote
`NameError`	Variable or function not defined (typo?)
`TypeError`	Wrong type, e.g. `"port" + 80` (use `f"port {80}"`)
`KeyError` / `IndexError`	Missing dict key / list index out of range
`ModuleNotFoundError`	Package not installed in this venv (`pip install ...`)
`FileNotFoundError`	Wrong path

REMEMBER

- Three habits: **virtual environment**, **timeout on every network call**, **secrets from the environment**.
- Return a non-zero exit code on failure so cron/CI notices.